ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

доцент, канд. техн. наук		А.В. Бржезовский
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №2

Транзакции, уровни изоляции и блокировки

по курсу: Методы и средства проектирования информационных систем и технологий

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4321		Г.В. Буренков
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

1 Цель работы 2
2 Задание 3
3 Ход работы 4
4 Выводы 7

1 Цель работы

Получить навык наблюдения аномалий параллельного доступа к базе варианта 5, их появления и исключения уровнями изоляции, чтения отчёта о блокировках и сообщения сервера о взаимоблокировке.

2 Задание

Двумя сеансами смоделировать грязное чтение, неповторяемое чтение и фантомные строки. Повторить опыты на уровнях read uncommitted, read committed, repeatable read и serializable. Пока одна транзакция держит изменённую строку, снять блокировки и пояснить режимы. Двумя транзакциями получить взаимоблокировку и привести текст, который вернул сервер.

3 Ход работы

Опыты идут в базе варианта 5. Для них использованы студент 100009 (Соколов Игорь Павлович) и студент 1 (Алексеев Иван Сергеевич) группы 4226, а также выступление 12010 с темой «Черновик темы для проверки изоляции». Каждый сеанс — отдельное соединение с сервером. На рисунке 1 представлен запрос для подготовки проверки изоляции.

```sql
COMMIT;  -- сброс случайно открытой транзакции клиента

INSERT INTO "Студент" ("ID_группы", "ФИО")
SELECT g."ID_группы", 'Соколов Игорь Павлович'
FROM "Группа" g
JOIN "Факультет" f ON f."ID_факультета" = g."ID_факультета"
JOIN "Вуз" v ON v."ID_вуза" = f."ID_вуза"
WHERE v."Название" = 'ГУАП'
  AND g."Номер_группы" = '4226'
  AND NOT EXISTS (
    SELECT 1 FROM "Студент" WHERE "ФИО" = 'Соколов Игорь Павлович'
  );

INSERT INTO "Выступление" ("ID_программы", "Название_темы")
SELECT p."ID_программы", 'Черновик темы для проверки изоляции'
FROM "Конференция" k
JOIN "Программа_конференции" p ON p."ID_конференции" = k."ID_конференции"
WHERE k."Название" = 'Информатика'
  AND NOT EXISTS (
    SELECT 1 FROM "Выступление"
    WHERE "Название_темы" = 'Черновик темы для проверки изоляции'
  );

COMMIT;

-- узнать ID для опытов
SELECT "ID_студента", "ФИО" FROM "Студент"
WHERE "ФИО" IN ('Соколов Игорь Павлович', 'Алексеев Иван Сергеевич')
ORDER BY "ID_студента";

SELECT "ID_выступления", "Название_темы" FROM "Выступление"
WHERE "Название_темы" = 'Черновик темы для проверки изоляции';
```

Results | Chart

ID_выступления	Название_темы
12010	Черновик темы для проверки изоляции

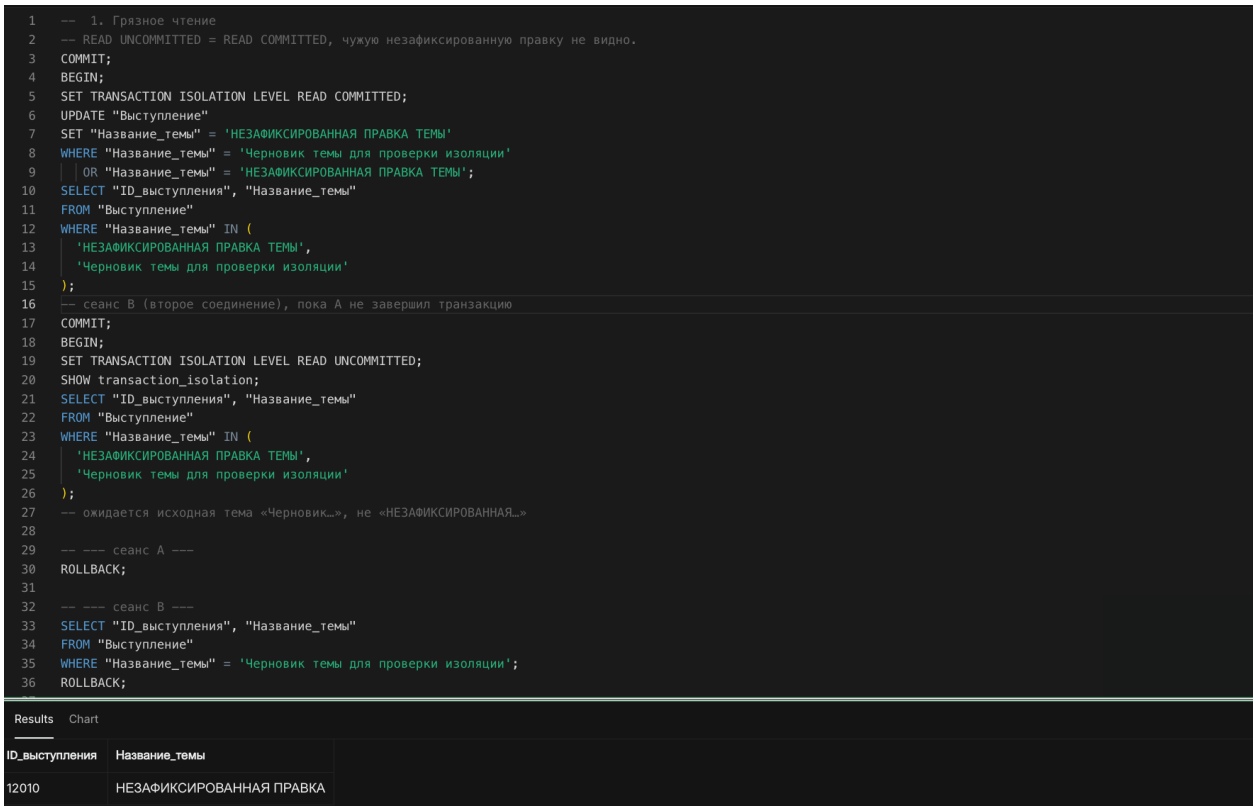
Рисунок 1 — запрос для подготовки проверки изоляции

3.1 Грязное чтение

Сеанс A в уровне read committed меняет тему на «НЕЗАФИКСИРОВАННАЯ ПРАВКА ТЕМЫ» и не фиксирует транзакцию. Сеанс B читает ту же строку:

```
BEGIN ISOLATION LEVEL READ UNCOMMITTED;
SELECT "Название_темы"
FROM "Выступление"
WHERE "ID_выступления" = 12010;
```

SHOW transaction_isolation в сеансе B вернул read uncommitted. Сеанс A уже видел «НЕЗАФИКСИРОВАННАЯ ПРАВКА ТЕМЫ», сеанс B прочитал «Черновик темы для проверки изоляции». Незафиксированной правки в ответе нет. После ROLLBACK сеанса A повторное чтение снова вернуло «Черновик темы для проверки изоляции». Грязное чтение не воспроизвелось: в PostgreSQL read uncommitted ведёт себя как read committed и показывает только зафиксированную версию строки. На рисунке 2 представлен запрос для грязного чтения.



```sql
--  1. Грязное чтение
-- READ UNCOMMITTED = READ COMMITTED, чужую незафиксированную правку не видно.
COMMIT;
BEGIN;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
UPDATE "Выступление"
SET "Название_темы" = 'НЕЗАФИКСИРОВАННАЯ ПРАВКА ТЕМЫ'
WHERE "Название_темы" = 'Черновик темы для проверки изоляции'
   OR "Название_темы" = 'НЕЗАФИКСИРОВАННАЯ ПРАВКА ТЕМЫ';
SELECT "ID_выступления", "Название_темы"
FROM "Выступление"
WHERE "Название_темы" IN (
  'НЕЗАФИКСИРОВАННАЯ ПРАВКА ТЕМЫ',
  'Черновик темы для проверки изоляции'
);
-- сеанс B (второе соединение), пока A не завершил транзакцию
COMMIT;
BEGIN;
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
SHOW transaction_isolation;
SELECT "ID_выступления", "Название_темы"
FROM "Выступление"
WHERE "Название_темы" IN (
  'НЕЗАФИКСИРОВАННАЯ ПРАВКА ТЕМЫ',
  'Черновик темы для проверки изоляции'
);
-- ожидается исходная тема «Черновик…», не «НЕЗАФИКСИРОВАННАЯ…»

-- --- сеанс A ---
ROLLBACK;

-- --- сеанс B ---
SELECT "ID_выступления", "Название_темы"
FROM "Выступление"
WHERE "Название_темы" = 'Черновик темы для проверки изоляции';
ROLLBACK;
```

Results Chart

ID_выступления	Название_темы
12010	НЕЗАФИКСИРОВАННАЯ ПРАВКА

Рисунок 2 — запрос для аналога грязного чтения

3.2 Неповторяемое чтение

Сеанс А дважды читает тему одного выступления. Между чтениями сеанс В меняет тему на «Черновик темы после фиксации правки» и выполняет COMMIT. Сеанс А:

```
SELECT "Название_темы"
FROM "Выступление"
WHERE "ID_выступления" = 12010;
```

На уровне read committed первое чтение вернуло «Черновик темы для проверки изоляции», второе — «Черновик темы после фиксации правки». Одна и та же строка в одной транзакции прочитана по-разному. На уровне repeatable read оба чтения вернули «Черновик темы для проверки изоляции»: зафиксированная правка второго сеанса в снимок первой транзакции не вошла.

На рисунке 3 представлен запрос для неповторяемого чтения.

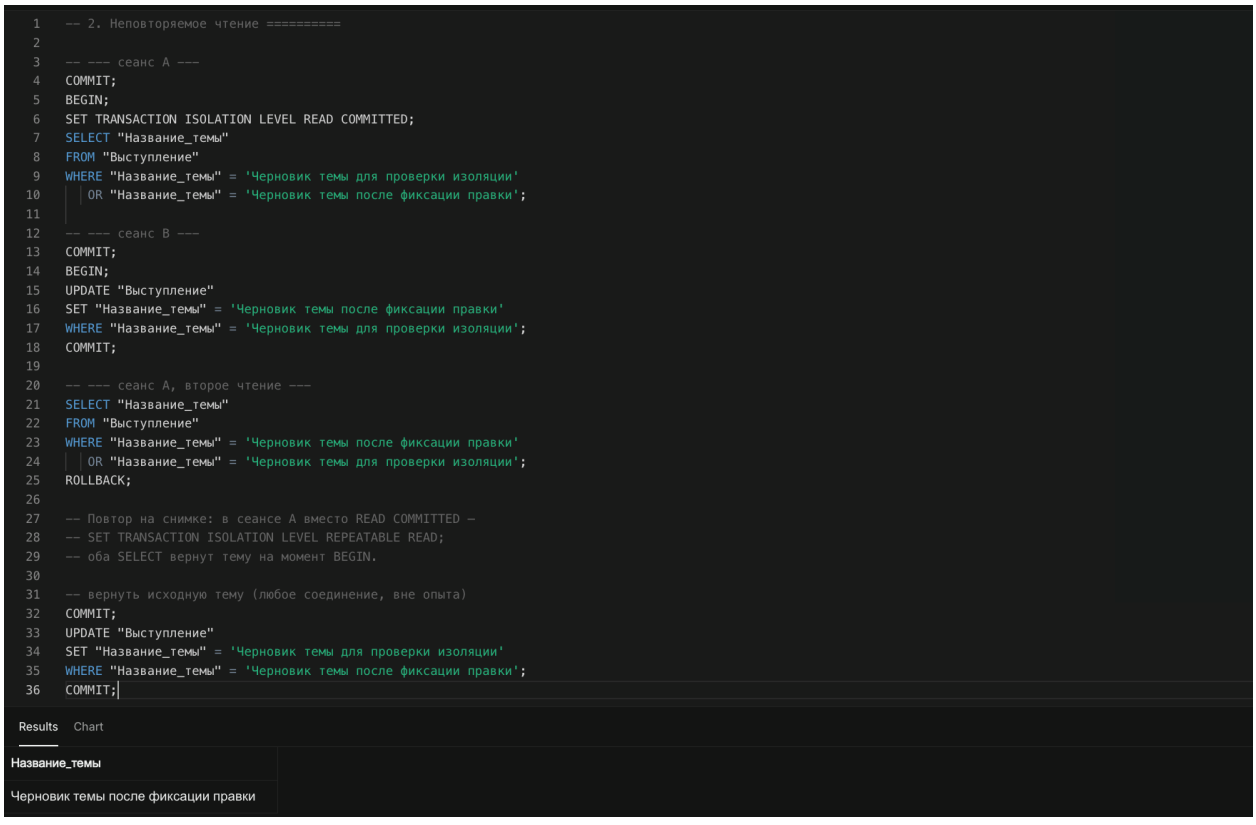


Рисунок 3 — запрос для неповторяемого чтения

3.3 Фантомные строки

Сеанс A дважды считает студентов группы 4226. Между подсчётами сеанс B вставляет новую строку и фиксирует её. После опыта вставленная строка удалена.

```
SELECT count(*)
FROM "Студент" s
JOIN "Группа" g
  ON g."ID_группы" = s."ID_группы"
WHERE g."Номер_группы" = '4226';
```

На уровне read committed первый подсчёт дал 3, второй — 4. Фантом воспроизведён, ошибки при COMMIT читающей транзакции не было. На repeatable read оба подсчёта равны 3 и 3, на serializable — 3 и 3. Фантом 3 затем 4 получен только на read committed: repeatable read уже держит снимок на начало транзакции и отсекает новую строку. На рисунке 4 для показа фантомных строк.

```
--  3. Фантомные строки ==========

-- --- сеанс A ---
COMMIT;
BEGIN;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT count(*)
FROM "Студент" s
JOIN "Группа" g ON g."ID_группы" = s."ID_группы"
WHERE g."Номер_группы" = '4226';

-- --- сеанс B ---
COMMIT;
BEGIN;
INSERT INTO "Студент" ("ID_группы", "ФИО")
SELECT g."ID_группы", 'Фантомный Студент Сеанса'
FROM "Группа" g
WHERE g."Номер_группы" = '4226';
COMMIT;

-- --- сеанс A, второй подсчёт ---
SELECT count(*)
FROM "Студент" s
JOIN "Группа" g ON g."ID_группы" = s."ID_группы"
WHERE g."Номер_группы" = '4226';
COMMIT;

-- убрать вставленную строку
DELETE FROM "Студент" WHERE "ФИО" = 'Фантомный Студент Сеанса';
COMMIT;
```

Results Chart

count
4

Рисунок 4 — запрос для показа фантомных строк

3.4 Блокировки

Сеанс-держатель (pid 10976) внутри открытой транзакции выполняет UPDATE студента 100009, присваивая ФИО текущее значение: строка блокируется, данные не меняются. Сеанс-ожидающий (pid 4556) читает ту же строку с FOR SHARE и останавливается. pg_blocking_pids вернул 10976.

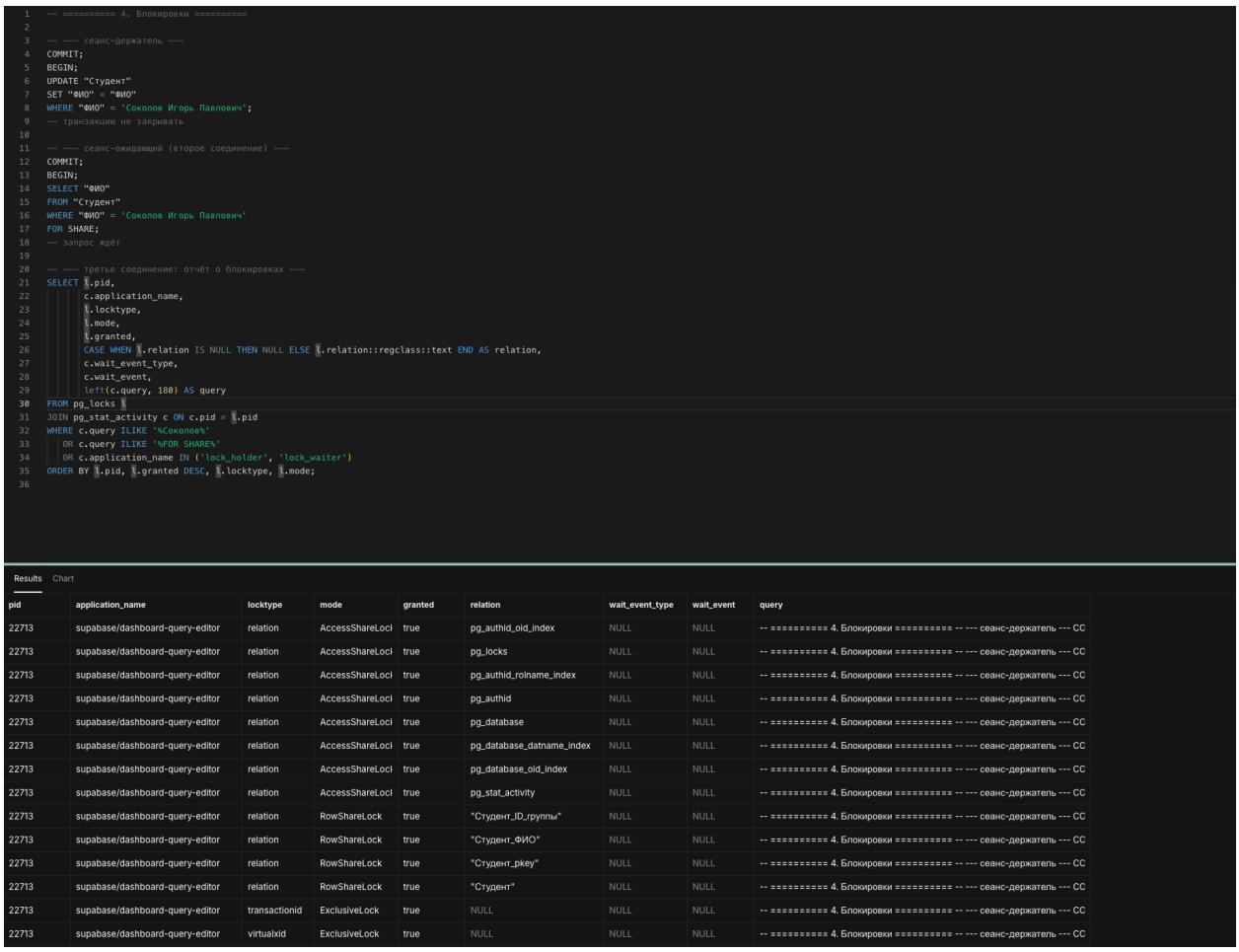
```
SELECT "ФИО"
FROM "Студент"
WHERE "ID_студента" = 100009
FOR SHARE;
```

В таблице 1 несколько строк отчёта о блокировках, не полный список. Пустой объект — блокировка идентификатора транзакции, а не таблицы.

Таблица 1 – Блокировки двух сеансов

Сеанс	Тип	Режим	Выдана	Объект	Ожидание
ожидающий	relation	RowShare Lock	да	Студент	Lock/ transactionid
ожидающий	tuple	RowShare Lock	да	Студент	Lock/ transactionid
ожидающий	transactionid	ShareLock	нет		Lock/ transactionid
держатель	relation	RowExclusive Lock	да	Студент	Client/ ClientRead
держатель	transactionid	Exclusive Lock	да		Client/ ClientRead

Держатель 10976 получил RowExclusiveLock на таблицу «Студент» и ExclusiveLock на идентификатор своей транзакции; он не ждёт. Ожидающий получил RowShareLock на таблицу и на строку, но ShareLock на транзакцию держателя не выдан: событие ожидания — Lock/transactionid. На рисунке 5 пример блокировки запроса.



```sql
-- ========== 4. Блокировки ==========

-- --- сеанс-держатель ---
COMMIT;
BEGIN;
UPDATE "Студент"
SET "ФИО" = "ФИО"
WHERE "ФИО" = 'Соколов Игорь Павлович';
-- транзакцию не закрывать

-- --- сеанс-ожидающий (второе соединение) ---
COMMIT;
BEGIN;
SELECT "ФИО"
FROM "Студент"
WHERE "ФИО" = 'Соколов Игорь Павлович'
FOR SHARE;
-- запрос ждёт

-- --- третье соединение: отчёт о блокировках ---
SELECT l.pid,
       c.application_name,
       l.locktype,
       l.mode,
       l.granted,
       CASE WHEN l.relation IS NULL THEN NULL ELSE l.relation::regclass::text END AS relation,
       c.wait_event_type,
       c.wait_event,
       left(c.query, 180) AS query
FROM pg_locks l
JOIN pg_stat_activity c ON c.pid = l.pid
WHERE c.query ILIKE '%Соколов%'
   OR c.query ILIKE '%FOR SHARE%'
   OR c.application_name IN ('lock_holder', 'lock_waiter')
ORDER BY l.pid, l.granted DESC, l.locktype, l.mode;
```

Results Chart

pid	application_name	locktype	mode	granted	relation	wait_event_type	wait_event	query
22713	supabase/dashboard-query-editor	relation	AccessShareLocl	true	pg_authid_oid_index	NULL	NULL	-- ========== 4. Блокировки ========== -- --- сеанс-держатель --- CC
22713	supabase/dashboard-query-editor	relation	AccessShareLocl	true	pg_locks	NULL	NULL	-- ========== 4. Блокировки ========== -- --- сеанс-держатель --- CC
22713	supabase/dashboard-query-editor	relation	AccessShareLocl	true	pg_authid_rolname_index	NULL	NULL	-- ========== 4. Блокировки ========== -- --- сеанс-держатель --- CC
22713	supabase/dashboard-query-editor	relation	AccessShareLocl	true	pg_authid	NULL	NULL	-- ========== 4. Блокировки ========== -- --- сеанс-держатель --- CC
22713	supabase/dashboard-query-editor	relation	AccessShareLocl	true	pg_database	NULL	NULL	-- ========== 4. Блокировки ========== -- --- сеанс-держатель --- CC
22713	supabase/dashboard-query-editor	relation	AccessShareLocl	true	pg_database_datname_index	NULL	NULL	-- ========== 4. Блокировки ========== -- --- сеанс-держатель --- CC
22713	supabase/dashboard-query-editor	relation	AccessShareLocl	true	pg_database_oid_index	NULL	NULL	-- ========== 4. Блокировки ========== -- --- сеанс-держатель --- CC
22713	supabase/dashboard-query-editor	relation	AccessShareLocl	true	pg_stat_activity	NULL	NULL	-- ========== 4. Блокировки ========== -- --- сеанс-держатель --- CC
22713	supabase/dashboard-query-editor	relation	RowShareLock	true	"Студент_ID_группы"	NULL	NULL	-- ========== 4. Блокировки ========== -- --- сеанс-держатель --- CC
22713	supabase/dashboard-query-editor	relation	RowShareLock	true	"Студент_ФИО"	NULL	NULL	-- ========== 4. Блокировки ========== -- --- сеанс-держатель --- CC
22713	supabase/dashboard-query-editor	relation	RowShareLock	true	"Студент_pkey"	NULL	NULL	-- ========== 4. Блокировки ========== -- --- сеанс-держатель --- CC
22713	supabase/dashboard-query-editor	relation	RowShareLock	true	"Студент"	NULL	NULL	-- ========== 4. Блокировки ========== -- --- сеанс-держатель --- CC
22713	supabase/dashboard-query-editor	transactionid	ExclusiveLock	true	NULL	NULL	NULL	-- ========== 4. Блокировки ========== -- --- сеанс-держатель --- CC
22713	supabase/dashboard-query-editor	virtualxid	ExclusiveLock	true	NULL	NULL	NULL	-- ========== 4. Блокировки ========== -- --- сеанс-держатель --- CC

Рисунок 5 - пример запроса блокировки сеанса

3.5 Взаимоблокировка

Два сеанса обновляют студентов 100009 и 1 в противоположном порядке. Присваивание ФИО самому себе берёт блокировку записи и не меняет имя. Сеанс 1 ниже идёт от первого студента ко второму; второй сеанс выполняет те же два оператора наоборот.

```
UPDATE "Студент" SET "ФИО" = "ФИО"
WHERE "ID_студента" = 100009;
UPDATE "Студент" SET "ФИО" = "ФИО"
WHERE "ID_студента" = 1;
```

Сеанс 1 зафиксировал транзакцию. Сеанс 2 получил отказ, его транзакция отменена. Сообщение сервера:

```
ERROR:  обнаружена взаимоблокировка
DETAIL:  Процесс 20500 ожидает в режиме ShareLock блокировку
"транзакция 881"; заблокирован процессом 24500.
```

Процесс 24500 ожидает в режиме ShareLock блокировку "транзакция 882"; заблокирован процессом 20500.

Сервер сообщает, что обнаружена взаимоблокировка: каждый из двух процессов ждёт ShareLock на транзакцию другого. Жертвой выбран один сеанс, второй после этого выполняет COMMIT. На рисунке 6 изображен пример взаимоблокировки.

```
1   --  5. Взаимоблокировка ==========
2   -- Сеанс 1 и сеанс 2 — разные соединения. Порог ожидания тупика — 1 с.
3
4   -- --- сеанс 1 ---
5   COMMIT;
6   BEGIN;
7   SET LOCAL deadlock_timeout = '1s';
8   UPDATE "Студент" SET "ФИО" = "ФИО" WHERE "ФИО" = 'Соколов Игорь Павлович';
9
10  -- --- сеанс 2 (до второго UPDATE сеанса 1) ---
11  COMMIT;
12  BEGIN;
13  SET LOCAL deadlock_timeout = '1s';
14  UPDATE "Студент" SET "ФИО" = "ФИО" WHERE "ФИО" = 'Алексеев Иван Сергеевич';
15
16  -- --- сеанс 1 ---
17  UPDATE "Студент" SET "ФИО" = "ФИО" WHERE "ФИО" = 'Алексеев Иван Сергеевич';
18
19  -- --- сеанс 2 ---
20  UPDATE "Студент" SET "ФИО" = "ФИО" WHERE "ФИО" = 'Соколов Игорь Павлович';
21  -- один сеанс: ошибка «обнаружена взаимоблокировка», второй — COMMIT;
```

Рисунок 6 - взаимоблокировки в сеансе

4 Выводы

Грязное чтение не получено. Уровень сеанса был read uncommitted, SHOW вернул read uncommitted, но SELECT увидел зафиксированную тему «Черновик темы для проверки изоляции», а не незафиксированную правку. В PostgreSQL read uncommitted ведёт себя как read committed. Неповторяемое чтение на read committed воспроизведено: второе чтение вернуло «Черновик темы после фиксации правки». На repeatable read оба чтения вернули «Черновик темы для проверки изоляции». Фантом 3 затем 4 получен только на read committed (3 и 4). На repeatable read и serializable повторный подсчёт остался равен 3. Пока сеанс 10976 держал студента 100009, сеанс 4556 с FOR SHARE ждал ShareLock. Встречные UPDATE дали сообщение «обнаружена взаимоблокировка»: сеанс 1 зафиксирован, сеанс 2 отменён.